المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍ - ⴱⴰⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport des travaux pratiques

Architecture Monolithique, Monolithe Modulaire et architecture distribuée

Filière : Intelligence Artificielle et Cybersécurité

Module :
Architecture Logicielle

Réalisé par :

Abderrahman MACHHOURI

Encadré par :
Pr. BE EL BAGHAZAOUI

Année universitaire : 2025-2026

Table des matières

Liste des figures **3**

Liste des tableaux **4**

1 TP1 : Application Monolithique Simple **5**
1.1 Objectifs 5
1.2 Architecture du Projet 5
1.3 Lien vers Github 5
1.4 Configuration 6
1.4.1 application.properties 6
1.5 Implémentation 6
1.5.1 Entité Product 6
1.5.2 Repository 6
1.5.3 Service 6
1.5.4 Controller 7
1.6 Tests avec Postman 8
1.6.1 Lister tous les produits (GET) 9
1.6.2 Créer un produit (POST) 9
1.6.3 Modifier un produit (PUT) 10
1.6.4 Supprimer un produit (DELETE) 10
1.7 Exercice 10
1.7.1 L'ajout de l'entité Category 10
1.7.2 Test avec Postman 11
1.8 Avantages et Limites du Monolithe 12
1.8.1 Avantages 12
1.8.2 Limites 12
1.9 Conclusion Globale 13

2 TP2 : Monolithe Modulaire **14**
2.1 Objectifs 14
2.2 Structure du TP 14
2.3 Lien vers Github 14
2.4 Diagramme des Dépendances 15
2.5 Tests avec Postman des trois modules 15
2.5.1 Module Product 15
2.5.2 Module Customer 17
2.5.3 Module Ordre 18
2.6 Vérification des dépendances 21
2.6.1 Création d'une commande avec un client inexistant 21

2.6.2 Création d'une commande avec un produit inexistant : 21
2.7 Conclusion Globale 22

3 TP3 : Architecture Distribuée avec Node.js et Docker 23
3.1 Objectifs 23
3.2 Architecture cible du système 23
3.3 Lien vers le code 23
3.4 Structure du projet 24
3.5 Flux de données 24
3.6 Lancement du système complet 24
3.6.1 Démarrer tous les conteneurs 24
3.6.2 Vérifier que tout est actif 25
3.7 Tests avec Postman 27
3.7.1 Test 1 : Créer une commande (POST) 27
3.7.2 Test 2 : Lister les commandes (GET) 28
3.7.3 Test 3 : Load balancing 29
3.8 Questions de compte-rendu 32

Table des figures

1.1 Architecture en couches du TP1 5
1.2 GET - Lister tous les produits 9
1.3 POST - Créer un produit 9
1.4 PUT - Modifier un produit 10
1.5 DELETE - Supprimer un produit 10
1.6 Relation ManyToOne entre Product et Category 11
1.7 POST - Créer un produit avec sa catégorie 12
1.8 GET - Récupérer un produit avec sa catégorie 12

2.1 Structure du projet modulaire 14
2.2 POST - Créer un produit (Module Product) 15
2.3 GET - Lister tous les produits (Module Product) 15
2.4 GET - Récupérer un produit par ID (Module Product) 16
2.5 PUT - Modifier un produit (Module Product) 16
2.6 DELETE - Supprimer un produit (Module Product) 17
2.7 POST - Créer un client (Module Customer) 17
2.8 GET - Lister tous les clients (Module Customer) 18
2.9 GET - Récupérer un client par ID (Module Customer) 18
2.10 POST - Créer une commande (Module Order) 19
2.11 GET - Lister toutes les commandes (Module Order) 19
2.12 GET - Détails d'une commande par ID (Module Order) 20
2.13 GET - Historique des commandes d'un client 20
2.14 DELETE - Supprimer une commande (Module Order) 21
2.15 Erreur lors de la création d'une commande avec un client inexistant 21
2.16 Erreur lors de la création d'une commande avec un produit inexistant . . . 22

3.1 Lancement de tous les conteneurs avec Docker Compose 25
3.2 Vérification Nginx + Node.js via localhost/health 25
3.3 Vérification directe de l'Instance 1 (port 3001) 26
3.4 Vérification directe de l'Instance 2 (port 3002) 26
3.5 Interface web de gestion RabbitMQ 27
3.6 POST - Créer une commande (réponse 201 Created) 27
3.7 GET - Premier appel, données servies depuis MongoDB 29
3.8 GET - Deuxième appel, données servies depuis le cache Redis 29
3.9 Load Balancing - Requête 1 traitée par Instance-1 30
3.10 Load Balancing - Requête 2 traitée par Instance-2 30
3.11 Load Balancing - Requête 3 traitée par Instance-1 31
3.12 Load Balancing - Requête 4 traitée par Instance-2 32

Liste des tableaux

3.1 Composants de l'architecture distribuée 23

TP1 : Application Monolithique Simple

1.1 Objectifs

— Créer une application Spring Boot monolithique
— Développer une API REST CRUD complète
— Comprendre l'architecture en couches
— Tester les endpoints avec Postman

1.2 Architecture du Projet

L'application suit une architecture en couches :
— **Entity** : Représentation de la base de données
— **Repository** : Accès aux données
— **Service** : Logique métier
— **Controller** : Exposition des endpoints REST

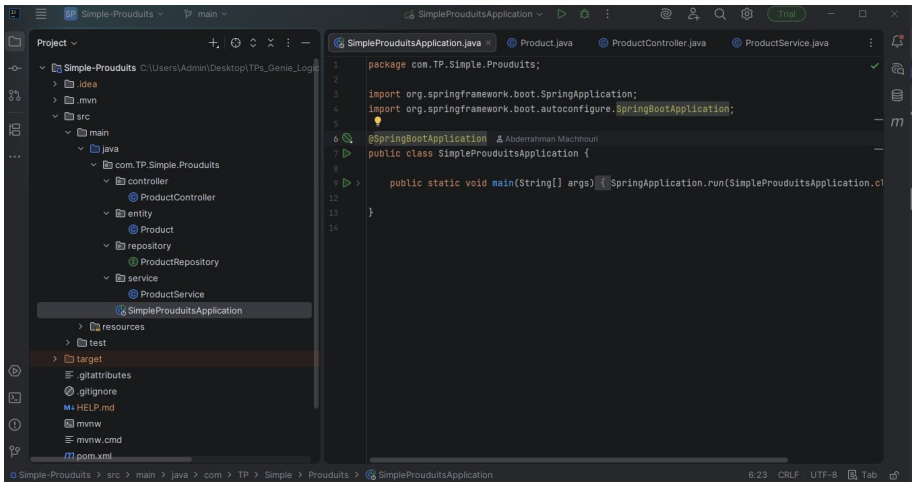

FIGURE 1.1 – Architecture en couches du TP1

1.3 Lien vers Github

Le code source de cette architecture est disponible sur GitHub :
`https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP1_Monolithe_Simple`

Et pour l'exercice :
https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP1_Monolithe_Simple_Exercice

1.4 Configuration

1.4.1 application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

1.5 Implémentation

1.5.1 Entité Product

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String description;
    private BigDecimal price;
    private Integer stock;
}
```

1.5.2 Repository

```
public interface ProductRepository
        extends JpaRepository<Product, Long> {

    List<Product> findByNameContainingIgnoreCase(String name);
}
```

1.5.3 Service

Le service contient la logique métier : création, modification, suppression.

```
package com.TP.Simple.Prouduits.service;
import com.TP.Simple.Prouduits.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
```

```java
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import com.TP.Simple.Prouduits.entity.Product;
@Service
@RequiredArgsConstructor
@Transactional
public class ProductService {
    private final ProductRepository ProductRepository;
    @Transactional(readOnly = true)
    public List<Product> getAll() {
        return ProductRepository.findAll();
    }
    @Transactional(readOnly = true)
    public Product getById(Long id) {
        return ProductRepository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException(
                        "Product not found with id: " + id));
    }
    public Product create(Product product) {
        return ProductRepository.save(product);
    }
    public Product update(Long id, Product details) {
        Product product = getById(id);
        product.setName(details.getName());
        product.setDescription(details.getDescription());
        product.setPrice(details.getPrice());
        product.setStock(details.getStock());
        return product;
    }
    public void delete(Long id) {
        Product product = getById(id);
        ProductRepository.delete(product);
    }
}
```

1.5.4 Controller

Les endpoints exposés :

- GET /api/products
- POST /api/products
- PUT /api/products/id
- DELETE /api/products/id

```java
package com.TP.Simple.Prouduits.controller;
import com.TP.Simple.Prouduits.entity.Product;
import com.TP.Simple.Prouduits.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
```

```java
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import java.util.List;
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {
    private final ProductService ProductService;
    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        List<Product> products = ProductService.getAll();
        return ResponseEntity.ok(products);
    }
    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable Long id) {
        Product product = ProductService.getById(id);
        return ResponseEntity.ok(product);
    }
    @PostMapping
    public ResponseEntity<Product> create(@Valid @RequestBody
       Product product) {
        Product created = ProductService.create(product);
        return ResponseEntity.status(HttpStatus.CREATED).body(
           created);
    }
    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable Long id,
                                          @Valid @RequestBody
                                             Product product) {
        Product updated = ProductService.update(id, product);
        return ResponseEntity.ok(updated);
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        ProductService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

1.6 Tests avec Postman

Les tests ont permis de vérifier :

- Création d'un produit
- Modification
- Suppression
- Validation des champs

1.6.1 Lister tous les produits (GET)

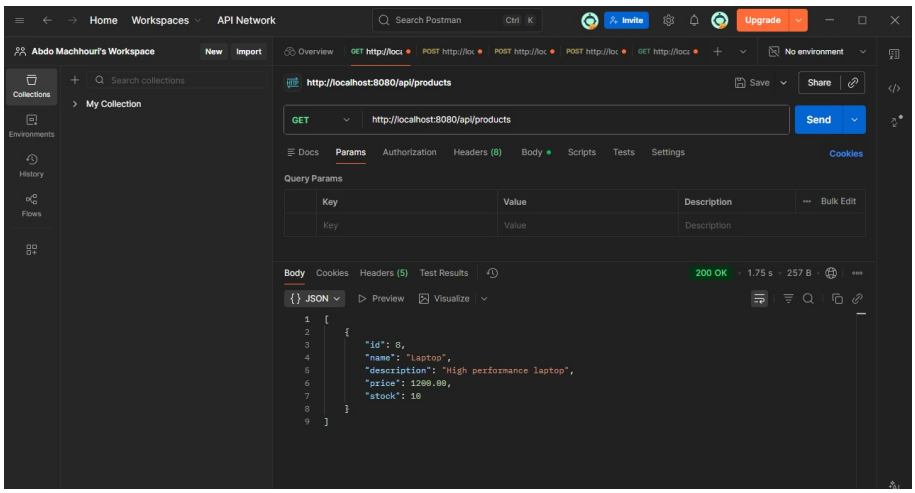

FIGURE 1.2 – GET - Lister tous les produits

1.6.2 Créer un produit (POST)

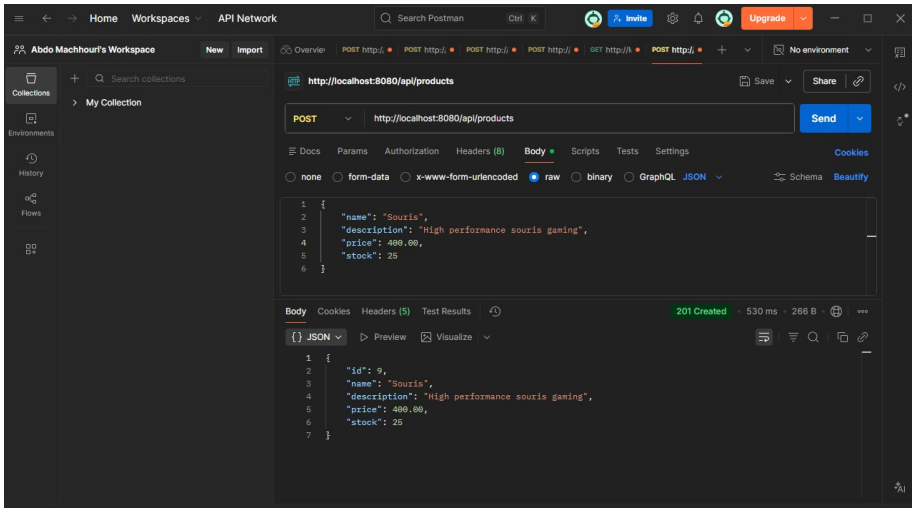

FIGURE 1.3 – POST - Créer un produit

1.6.3 Modifier un produit (PUT)

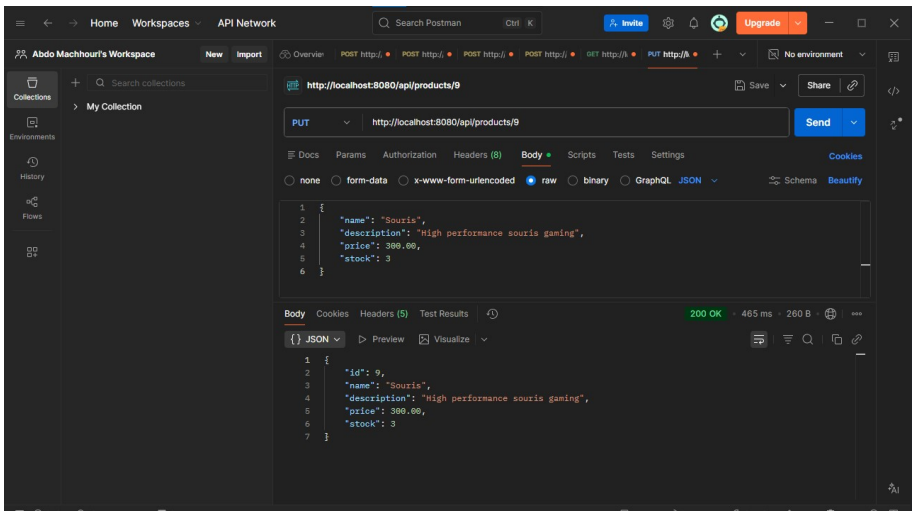

FIGURE 1.4 – PUT - Modifier un produit

1.6.4 Supprimer un produit (DELETE)

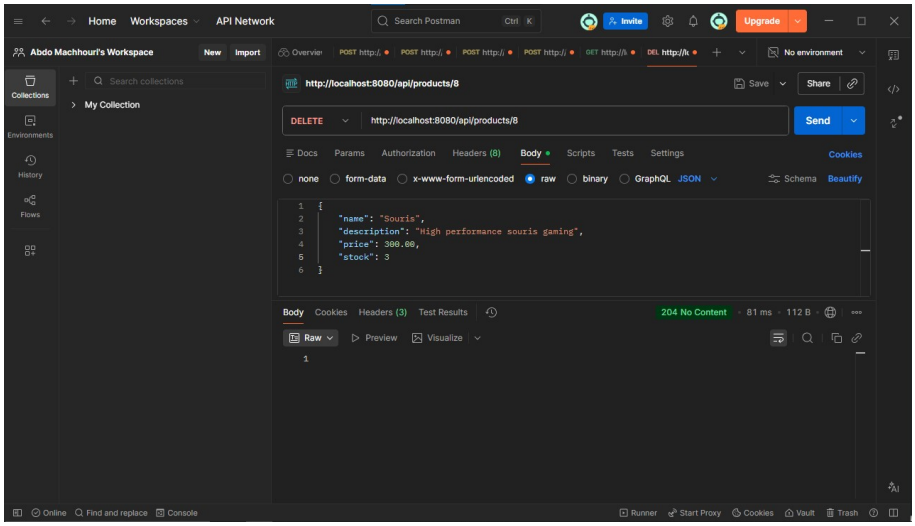

FIGURE 1.5 – DELETE - Supprimer un produit

1.7 Exercice

1.7.1 L'ajout de l'entité Category

On va ajouter l'entité Category et on crée la relation ManyToOne avec l'entité Product.

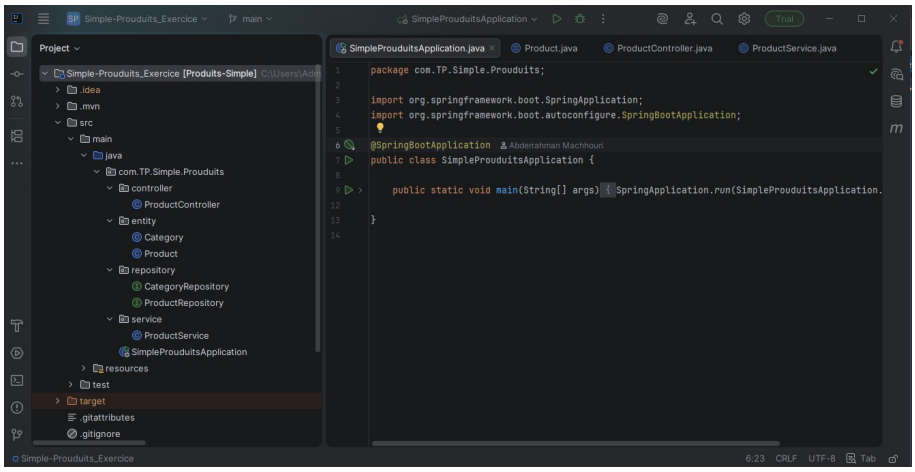

FIGURE 1.6 – Relation ManyToOne entre Product et Category

```
package com.TP.Simple.Prouduits.entity;
import jakarta.persistence.*;
import lombok.*;
@Entity
@Table(name = "category")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name_category;
    @Column(length = 1000)
    private String description;
}
```

1.7.2 Test avec Postman

Dans ce test on va ajouter un produit avec sa catégorie et on va voir si ca marche ou pas.
Maintenant on va faire la méthode POST :

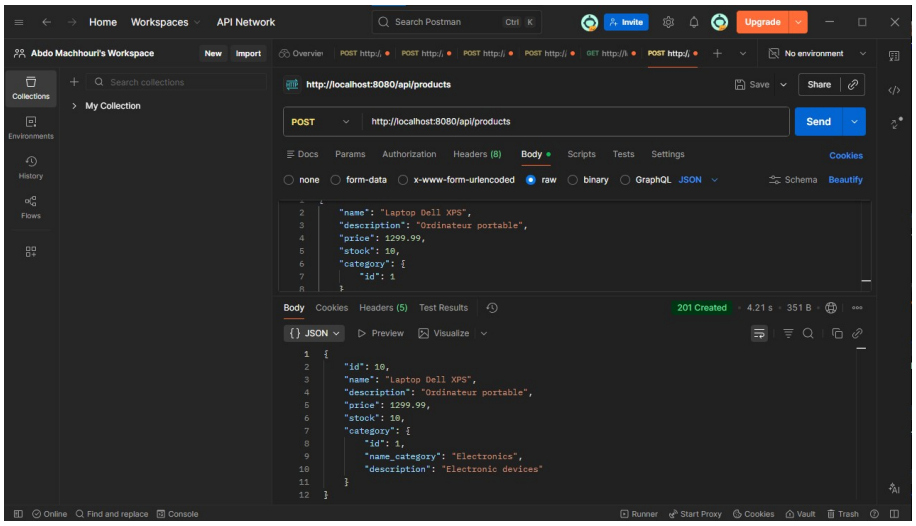

FIGURE 1.7 – POST - Créer un produit avec sa catégorie

Maintenant on va récupérer le produit avec la méthode GET pour voir sa categorie.

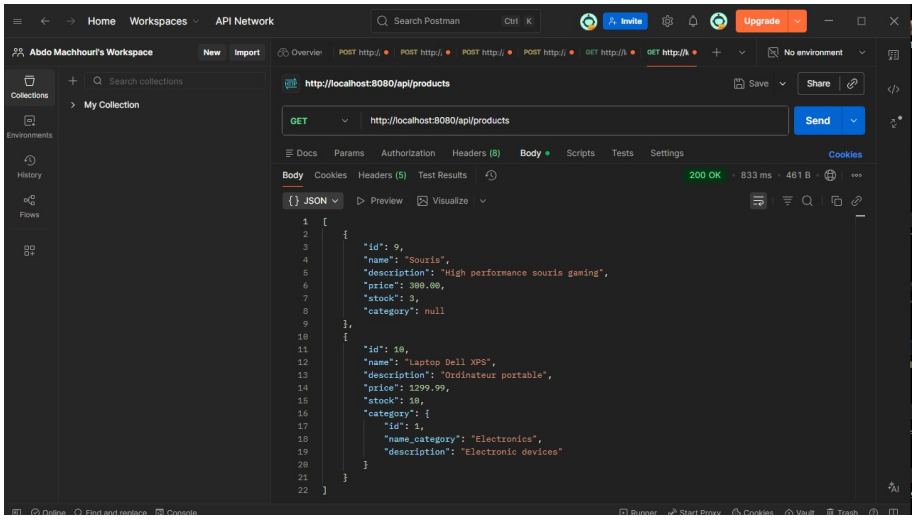

FIGURE 1.8 – GET - Récupérer un produit avec sa catégorie

Et voila on voit bien la catégorie de notre produit.

1.8 Avantages et Limites du Monolithe

1.8.1 Avantages

— Simple à développer
— Déploiement facile
— Performances élevées

1.8.2 Limites

— Difficulté de maintenance à grande échelle

— Couplage fort entre modules
— Scalabilité limitée

1.9 Conclusion Globale

Ce TP a permis de mettre en place une application monolithique simple avec Spring Boot, en suivant une architecture en couches claire (Entity, Repository, Service, Controller). Nous avons développé une API REST complète pour la gestion de produits, en utilisant à la fois une approche classique et l'option @RepositoryRestResource qui génère automatiquement les endpoints. Les tests avec Postman ont validé le bon fonctionnement des opérations CRUD. L'exercice proposé (ajout d'une entité Category avec une relation ManyToOne vers Product) a illustré l'extension du modèle et l'enrichissement des associations. Ce travail met en évidence la simplicité de développement d'une architecture monolithique, mais aussi ses limites en termes de scalabilité et de maintenabilité à grande échelle, ouvrant la voie vers des architectures distribuées comme celle abordée dans les TP suivants.

TP2 : Monolithe Modulaire

2.1 Objectifs

— Restructurer le projet en modules métier
— Introduire les DTO
— Utiliser MapStruct
— Préparer une future migration microservices

2.2 Structure du TP

Le projet est divisé en trois modules :
— Module Product
— Module Customer
— Module Order

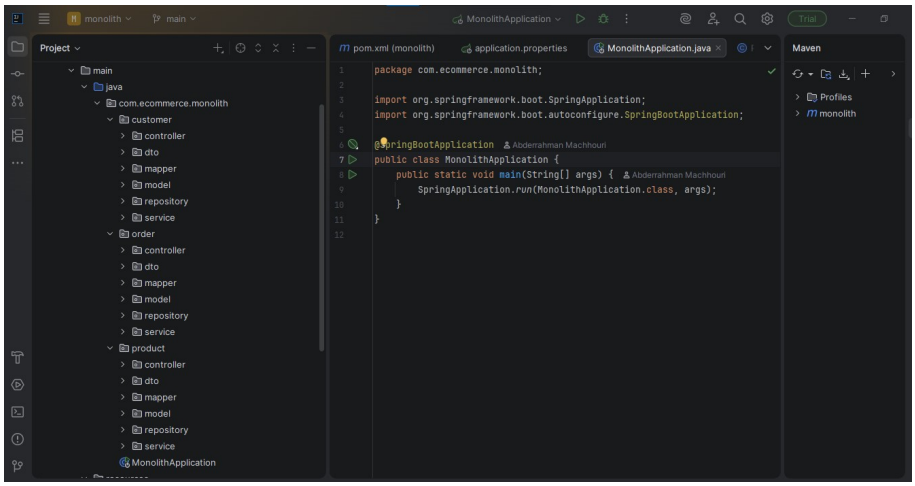

FIGURE 2.1 – Structure du projet modulaire

2.3 Lien vers Github

Donc chaque module contient six packages.
Pour voir le code source de tous les modules et de l'application, voici l'url correspondent : `https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP2_monolithe_modulaire/monolith`

2.4 Diagramme des Dépendances

— Product : indépendant
— Customer : indépendant
— Order : dépend de Product et Customer

2.5 Tests avec Postman des trois modules

2.5.1 Module Product

— La méthode POST pour créer un produit :

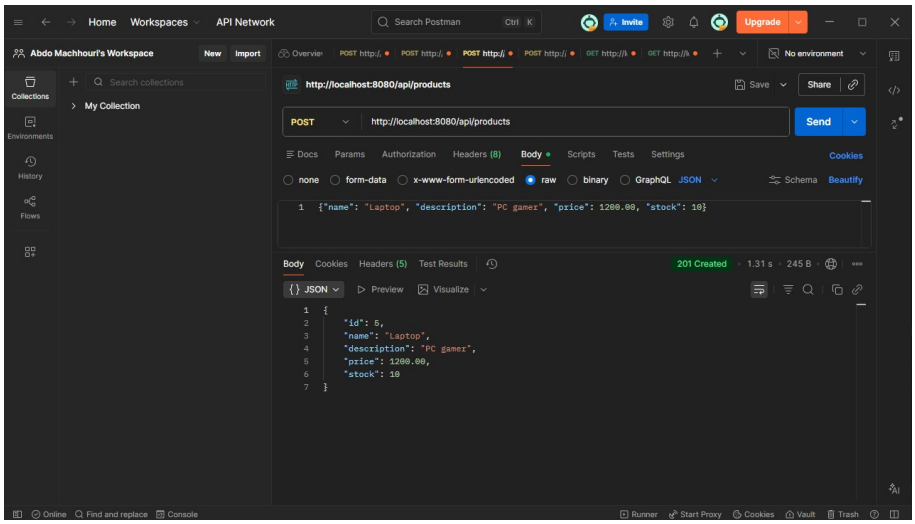

FIGURE 2.2 – POST - Créer un produit (Module Product)

— La méthode GET pour récuperer tous les produits et aussi par id :

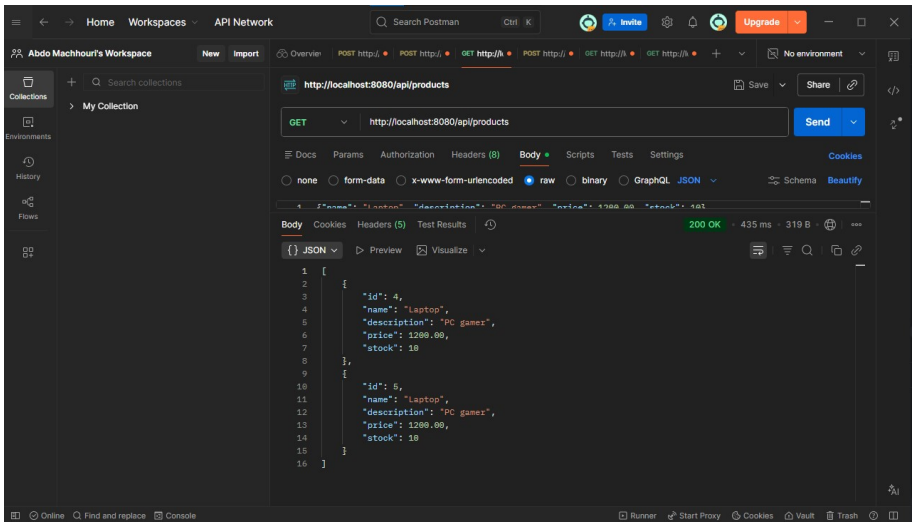

FIGURE 2.3 – GET - Lister tous les produits (Module Product)

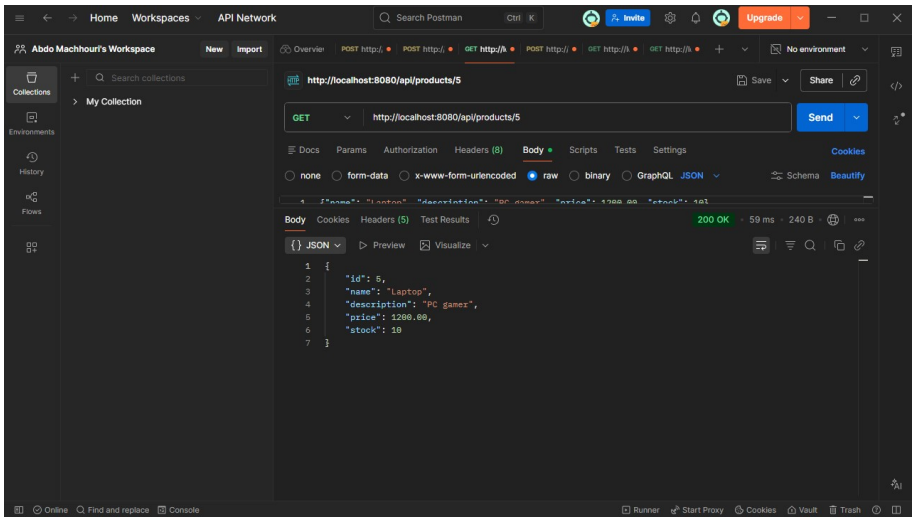

FIGURE 2.4 – GET - Récupérer un produit par ID (Module Product)

— La méthode PUT pour modifier un produit avec son id :

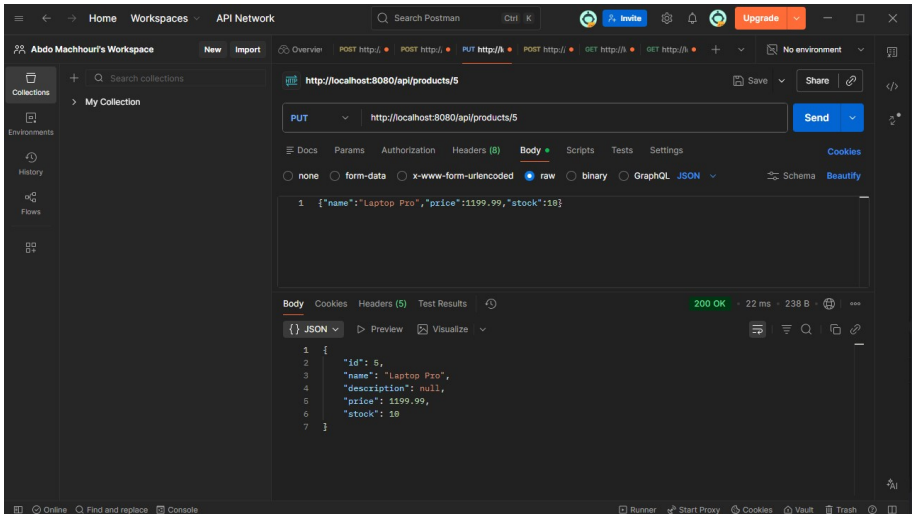

FIGURE 2.5 – PUT - Modifier un produit (Module Product)

— La méthode DELETE pour supprimer un produits avec son id :

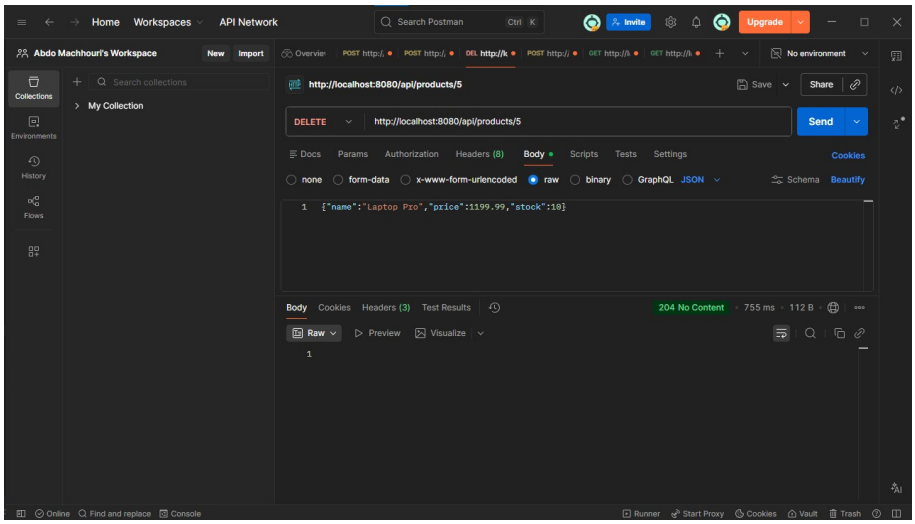

FIGURE 2.6 – DELETE - Supprimer un produit (Module Product)

2.5.2 Module Customer

— La méthode POST pour créer un client :

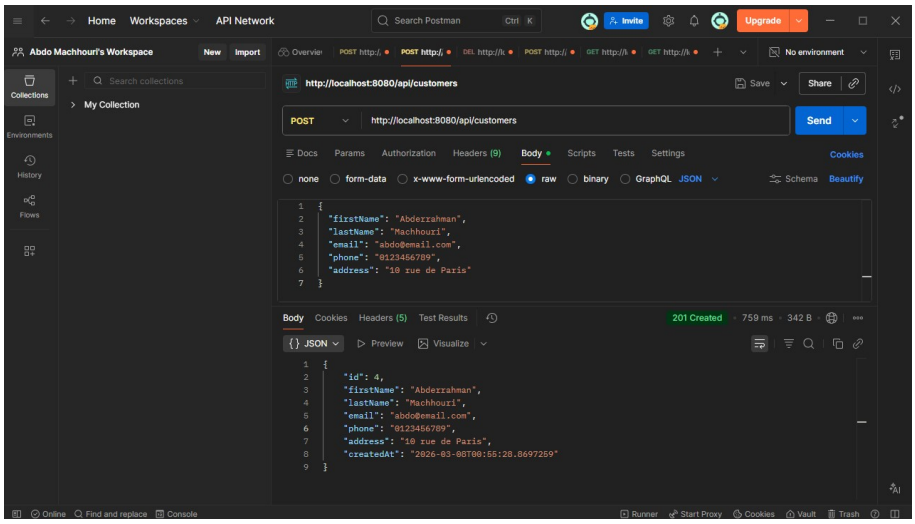

FIGURE 2.7 – POST - Créer un client (Module Customer)

— La méthode GET pour récuperer tous les clients :

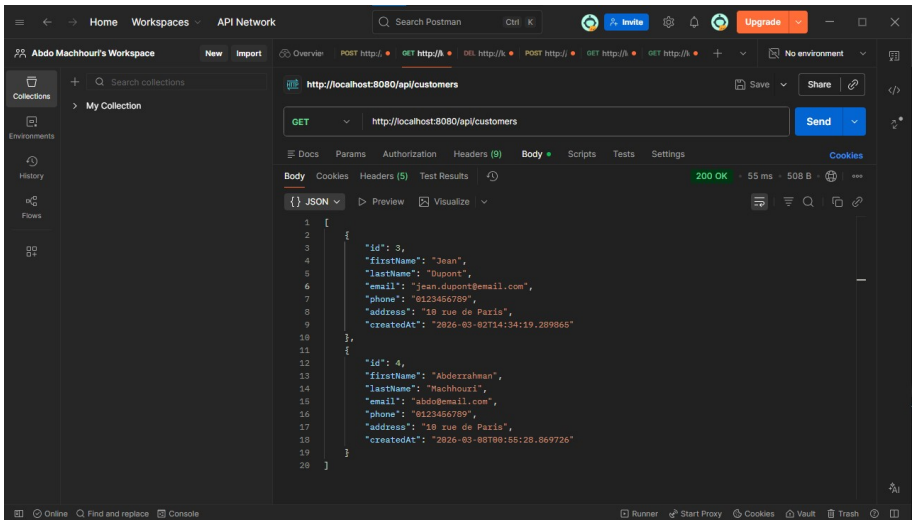

FIGURE 2.8 – GET - Lister tous les clients (Module Customer)

— La méthode GET pour récuperer un client en details :

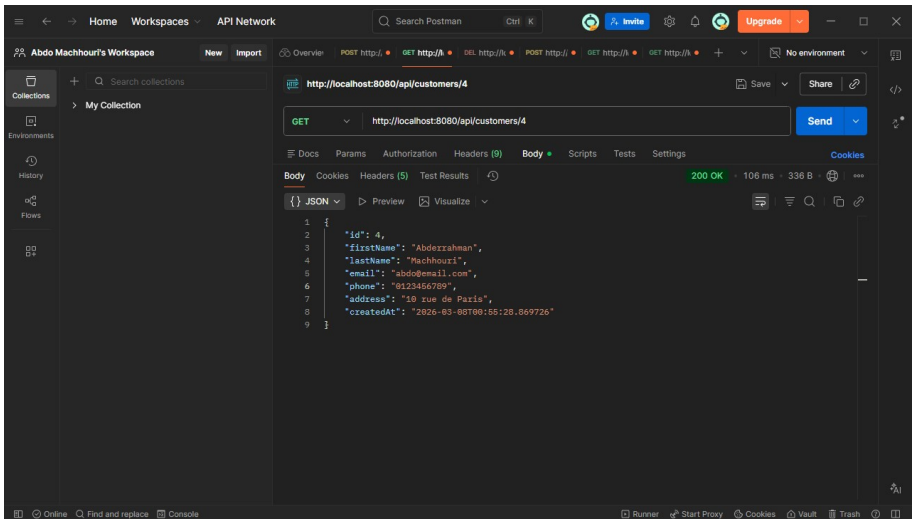

FIGURE 2.9 – GET - Récupérer un client par ID (Module Customer)

2.5.3 Module Ordre

— La méthode POST pour créer une commande contenant les infos sur le client et les produits :

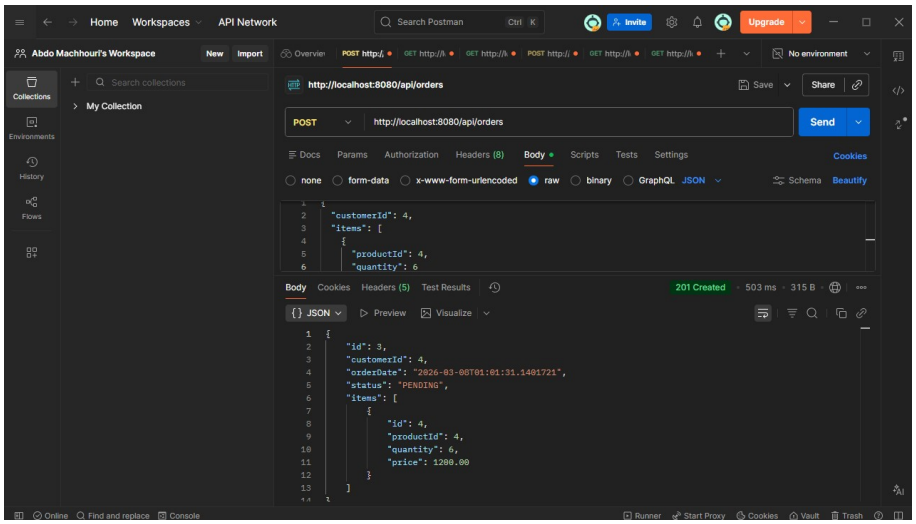

FIGURE 2.10 – POST - Créer une commande (Module Order)

— La méthode GET (/api/orders) pour lister les commandes :

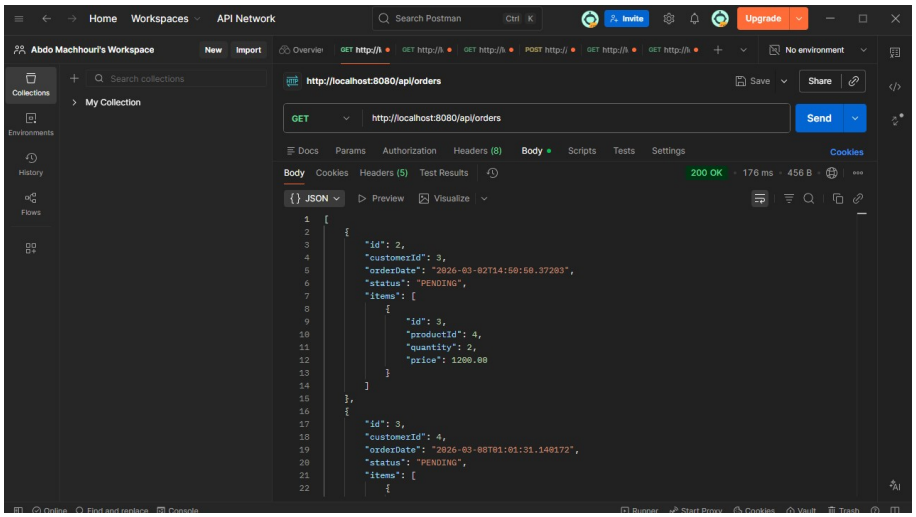

FIGURE 2.11 – GET - Lister toutes les commandes (Module Order)

— La méthode GET (/api/orders/id) pour lister les details d'une commande :

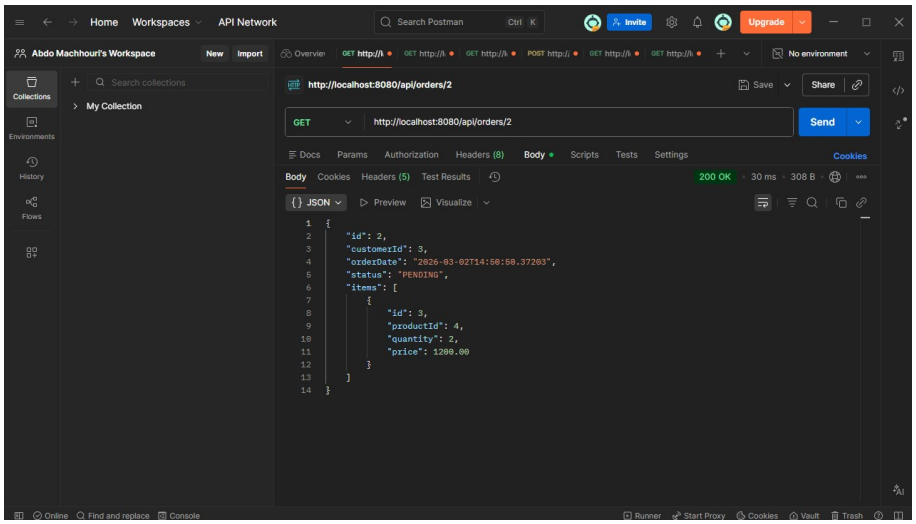

FIGURE 2.12 – GET - Détails d'une commande par ID (Module Order)

— La méthode GET (/api/customers/id/orders) pour voir l'historique des commandes d'un client :

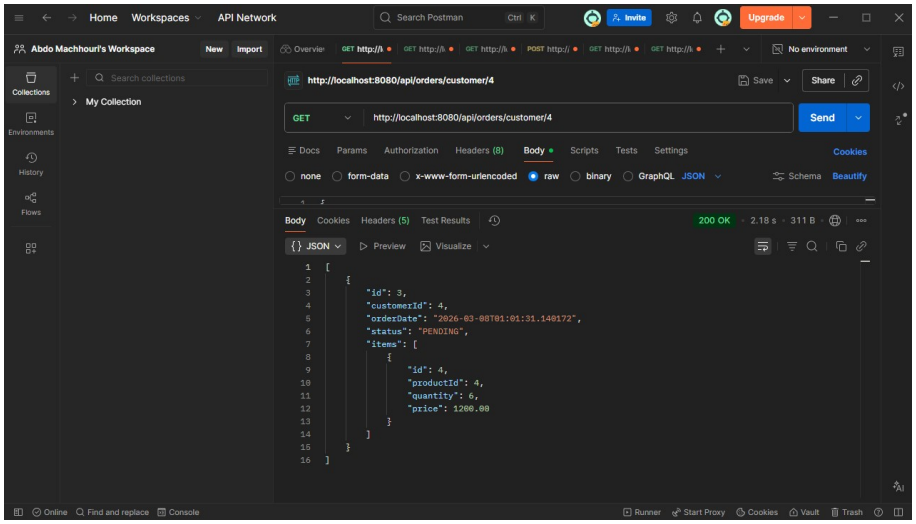

FIGURE 2.13 – GET - Historique des commandes d'un client

— La méthode DELETE (/api/orders/id) pour supprimer une commande :

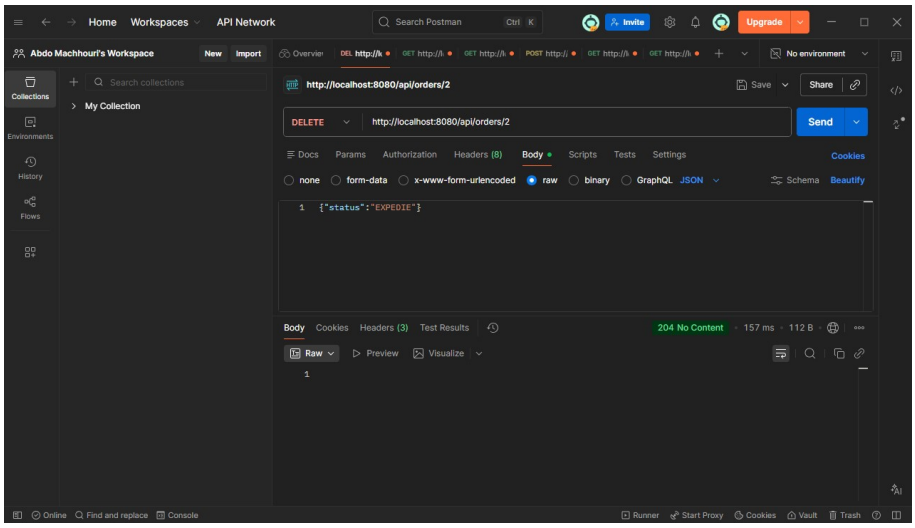

FIGURE 2.14 – DELETE - Supprimer une commande (Module Order)

2.6 Vérification des dépendances

2.6.1 Création d'une commande avec un client inexistant

On va créer une commande avec un client inexistant et il doit retourner 404 Not Found ou 400 Bad Request avec un message d'erreur explicite.

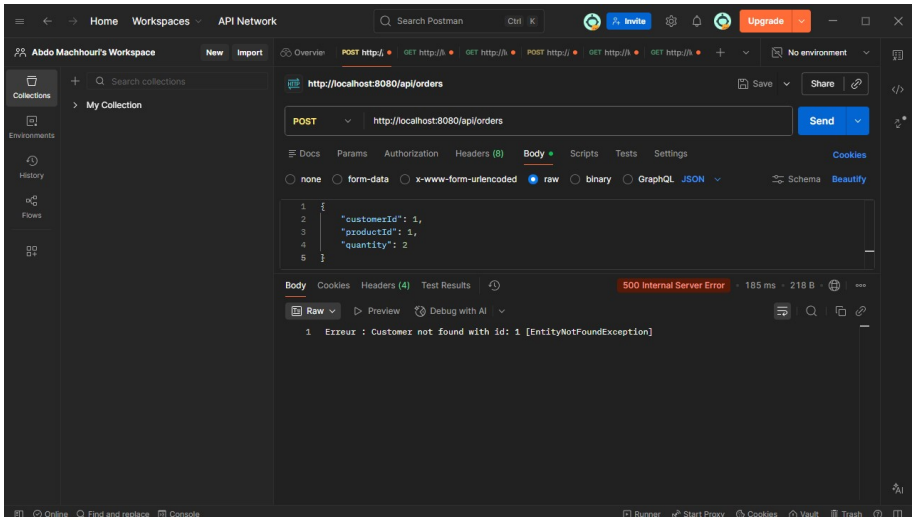

FIGURE 2.15 – Erreur lors de la création d'une commande avec un client inexistant

2.6.2 Création d'une commande avec un produit inexistant :

La meme chose on va créer une commande avec un produit inexsitant et on attend qu'il retourne un erreur.

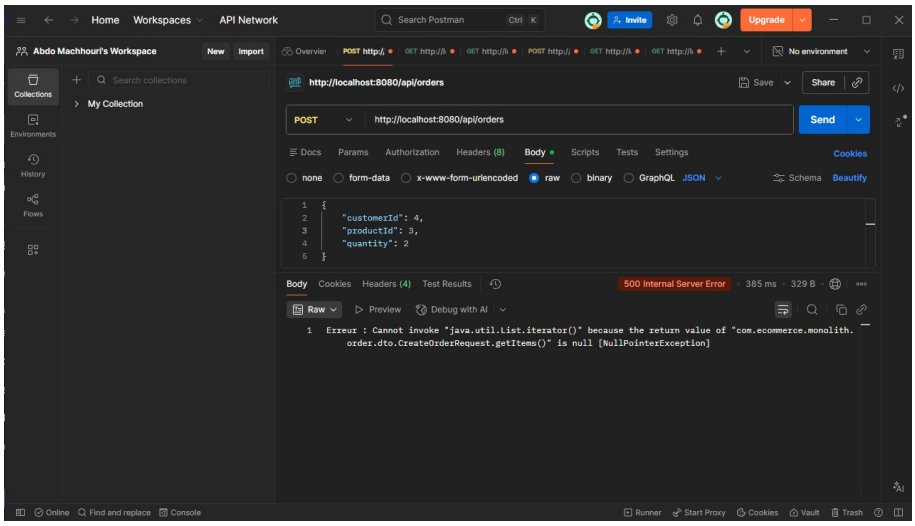

FIGURE 2.16 – Erreur lors de la création d'une commande avec un produit inexistant

2.7 Conclusion Globale

Ce deuxième TP a permis de faire évoluer l'application monolithique initiale vers une architecture modulaire, mieux structurée et plus proche des principes métier. En introduisant des modules distincts pour la gestion des produits, des clients et des commandes, nous avons appliqué une séparation claire des responsabilités. L'utilisation de DTOs et de mappers (avec MapStruct) a renforcé l'indépendance entre la couche de persistance et la couche de présentation, tandis que le recours à des interfaces de service a limité les dépendances directes entre modules. Les tests fonctionnels menés avec Postman ont validé le bon fonctionnement des endpoints, y compris les vérifications d'intégrité référentielle (existence d'un client ou d'un produit avant création d'une commande) et l'accès à l'historique des commandes d'un client. Cette restructuration constitue une étape préparatoire essentielle à une éventuelle migration vers une architecture microservices, en posant des bases modulaires, faiblement couplées et facilement extractibles.

TP3 : Architecture Distribuée avec Node.js et Docker

3.1 Objectifs

- Déployer une architecture distribuée complète avec plusieurs composants interconnectés.
- Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages.
- Implémenter un flux complet : POST commande → stockage MongoDB → cache Redis → notification RabbitMQ.
- Tester et observer le comportement distribué avec Postman.

3.2 Architecture cible du système

Table 3.1 – Composants de l'architecture distribuée

Composant	**Technologie**	**Rôle**
Client	Postman	Envoie les requêtes HTTP POST/GET pour créer et lister des commandes
Load Balancer	Nginx	Reçoit le trafic et le répartit entre 2 instances Node.js (port 3001 et 3002)
Serveur applicatif	Node.js / Express	Implémente la logique métier : validation, traitement et réponse
Base de données	MongoDB	Stocke toutes les commandes de façon persistante
Cache	Redis	Mémorise les résultats GET pour éviter des requêtes répétitives à MongoDB
Bus de messages	RabbitMQ	Reçoit un événement après chaque commande et affiche une notification

3.3 Lien vers le code

Le code source de cette architecture est disponible sur GitHub : `https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/tp-archi-distribuee`

3.4 Structure du projet

Le projet suit une architecture microservices orchestrée avec Docker Compose. Voici l'arborescence des fichiers principaux :

```
tp-archi-distribuee/
 docker-compose.yml        # orchestre tous les conteneurs
 nginx/
    nginx.conf            # configuration du load balancer
 app/
    package.json
    server.js             # serveur Node.js principal
    routes/
       commandes.js      # routes API
    services/
       mongodb.js        # connexion MongoDB
       redis.js          # connexion Redis
       rabbitmq.js       # connexion RabbitMQ
    consumer.js           # consommateur de messages
```

3.5 Flux de données

Postman → Nginx :80 → Node.js :3001 / :3002
Node.js → MongoDB (stockage persistant)
Node.js → Redis (cache GET /commands)
Node.js → RabbitMQ → Consommateur → Notification console

3.6 Lancement du système complet

3.6.1 Démarrer tous les conteneurs

Depuis la racine du projet, exécuter la commande suivante :

```
docker-compose up --build
```

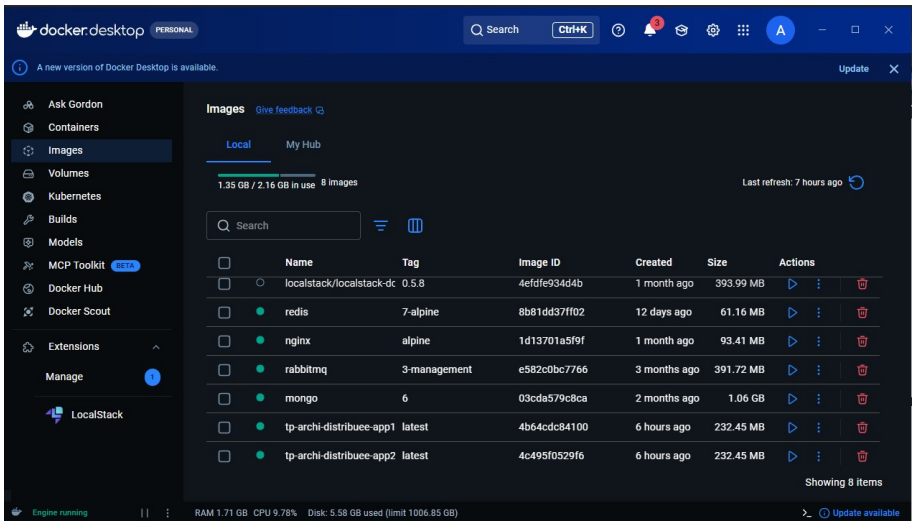

FIGURE 3.1 – Lancement de tous les conteneurs avec Docker Compose

3.6.2 Vérifier que tout est actif

Vérification Nginx + Node.js

URL : `http://localhost/health`

Résultat attendu :

```
{ status: 'OK', instance: 'Instance-1' }
```

ou

```
{ status: 'OK', instance: 'Instance-2' }
```

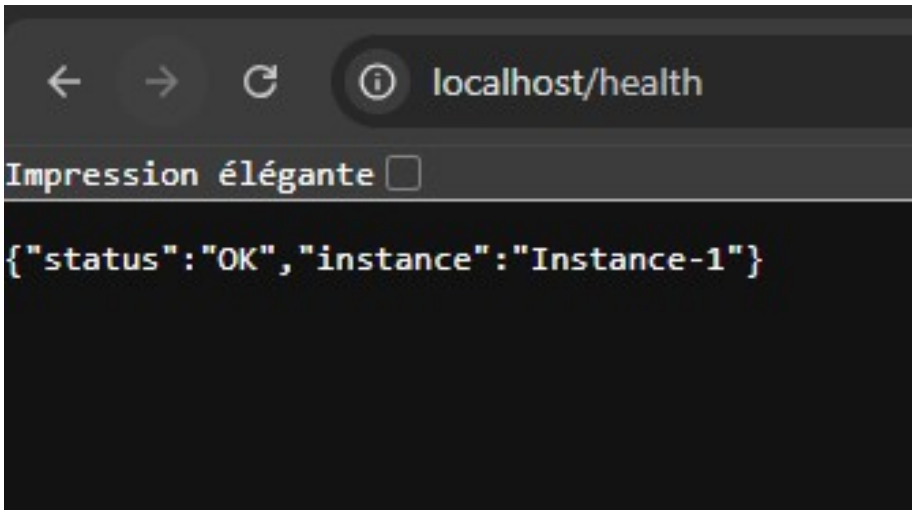

FIGURE 3.2 – Vérification Nginx + Node.js via localhost/health

Vérification Instance 1 directe

URL : `http://localhost:3001/health`
Résultat attendu :

```
{ status: 'OK', instance: 'Instance-1' }
```

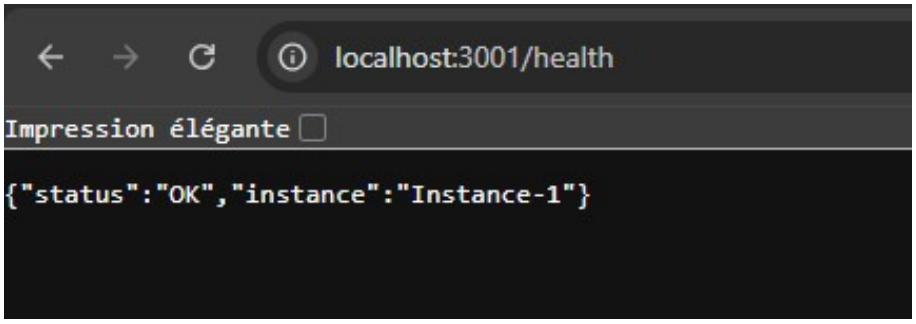

FIGURE 3.3 – Vérification directe de l'Instance 1 (port 3001)

Vérification Instance 2 directe

URL : `http://localhost:3002/health`
Résultat attendu :

```
{ status: 'OK', instance: 'Instance-2' }
```

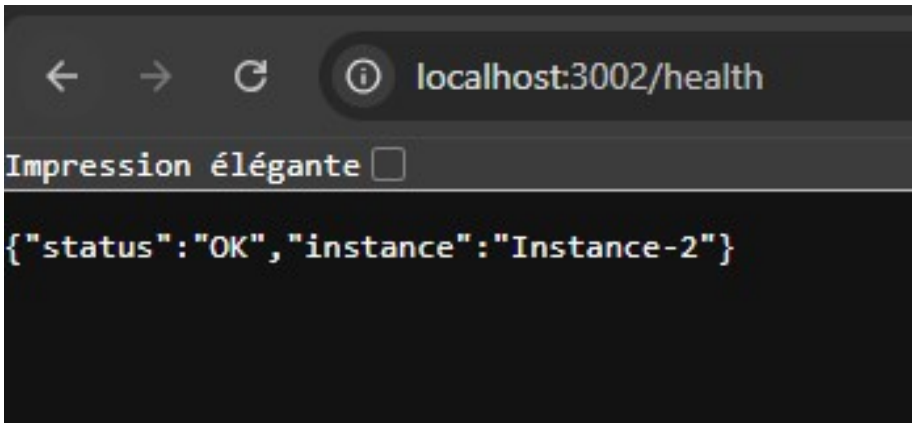

FIGURE 3.4 – Vérification directe de l'Instance 2 (port 3002)

Vérification RabbitMQ

URL : `http://localhost:15672`
Identifiants : admin / admin
Résultat attendu : Interface web de gestion RabbitMQ

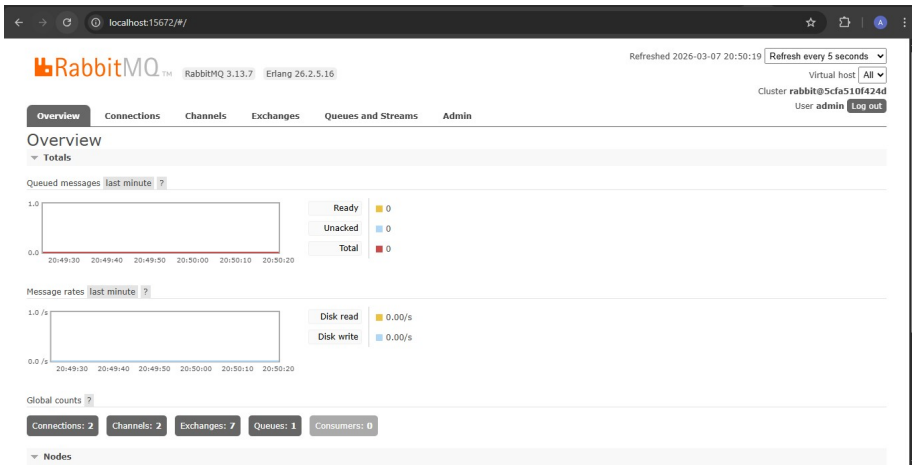

FIGURE 3.5 – Interface web de gestion RabbitMQ

3.7 Tests avec Postman

3.7.1 Test 1 : Créer une commande (POST)

- Méthode : POST
- URL : `http://localhost/commands`
- Headers : Content-Type : application/json
- Body :

```
{
  "produit": "Laptop Dell XPS",
  "quantite": 2,
  "client": "Ahmed Benali"
}
```

- Réponse attendue :

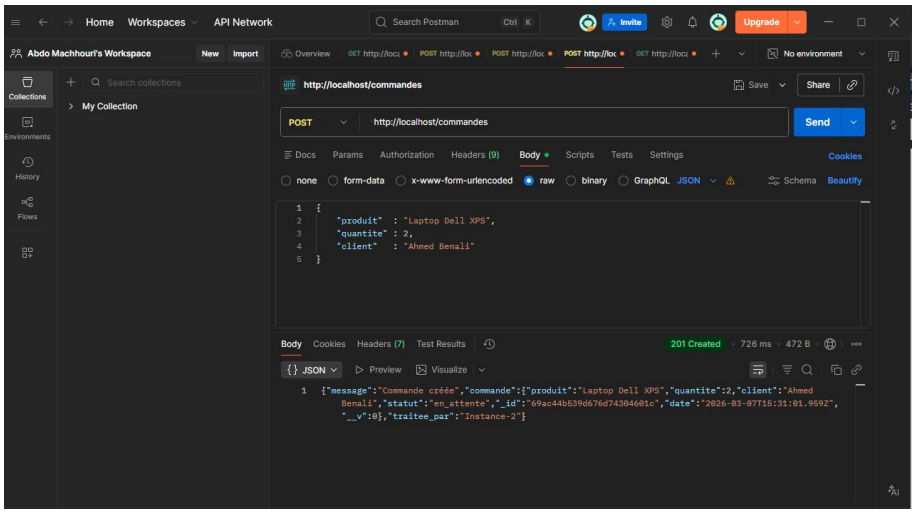

FIGURE 3.6 – POST - Créer une commande (réponse 201 Created)

Le serveur répond avec un code 201 Created, ce qui indique que la commande a été créée avec succès.

3.7.2 Test 2 : Lister les commandes (GET)

— Méthode : GET
— URL : `http://localhost/commands`
— Observation :

— 1er appel → source MongoDB :

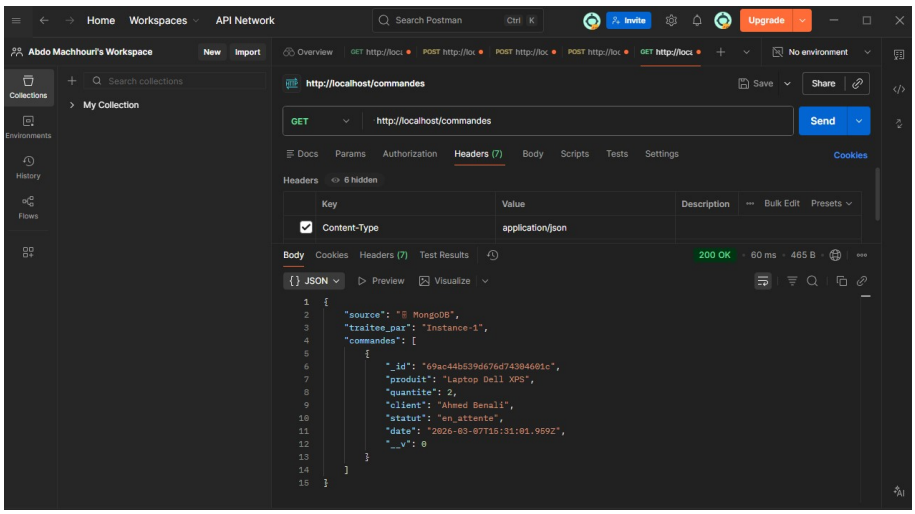

FIGURE 3.7 – GET - Premier appel, données servies depuis MongoDB

La vérification est réussie : la requête POST retourne un statut 200 OK avec des données JSON. La présence du champ "source" : "MongoDB" confirme que les données proviennent bien de la base MongoDB, ce qui est conforme à l'architecture attendue.

— 2ème appel → source Redis (cache) :

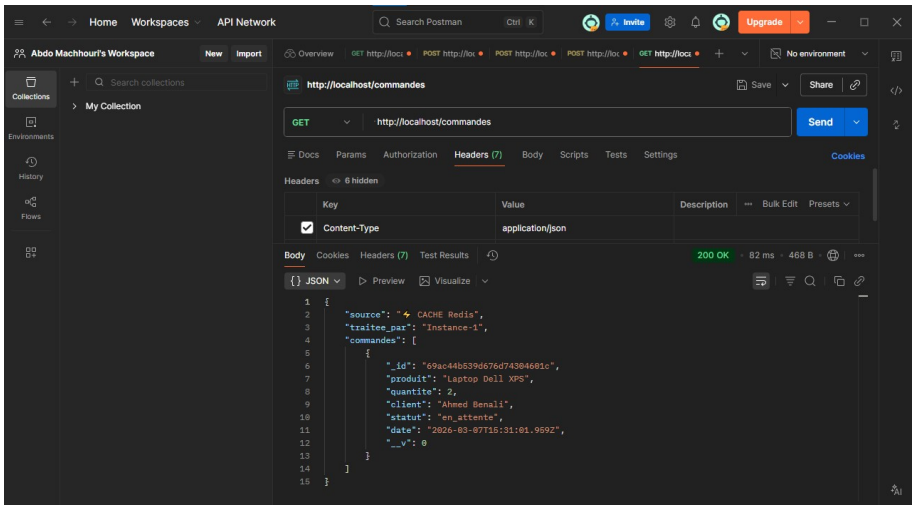

FIGURE 3.8 – GET - Deuxième appel, données servies depuis le cache Redis

La vérification est réussie : la requête POST retourne un statut 200 OK avec des données JSON. Le champ "source" : "CACHE Redis" confirme que les données ont été servies depuis le cache Redis, ce qui valide le bon fonctionnement du mécanisme de mise en cache.

3.7.3 Test 3 : Load balancing

Envoyer 4 requêtes POST successives et observer le champ `traitee_par` :

— Requête 1 → traitee_par : Instance-1

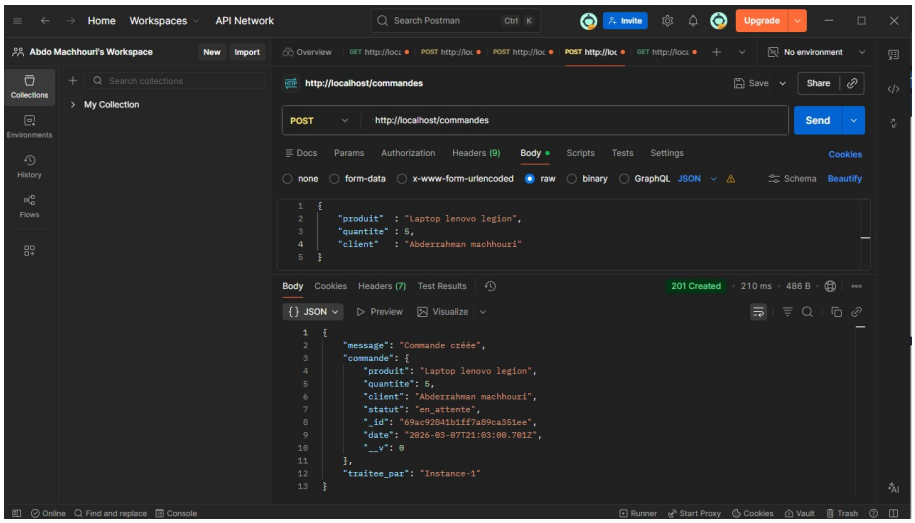

FIGURE 3.9 – Load Balancing - Requête 1 traitée par Instance-1

— Requête 2 → traitee_par : Instance-2

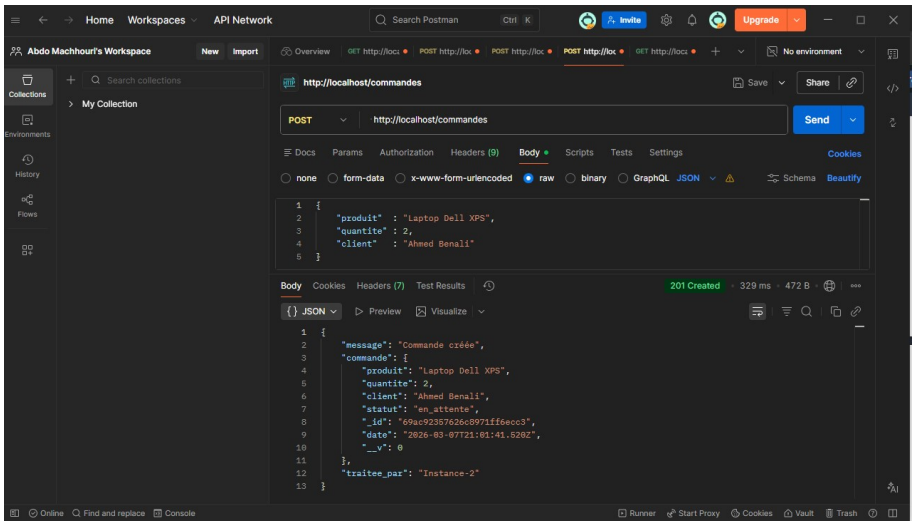

FIGURE 3.10 – Load Balancing - Requête 2 traitée par Instance-2

— Requête 3 → traitee_par : Instance-1

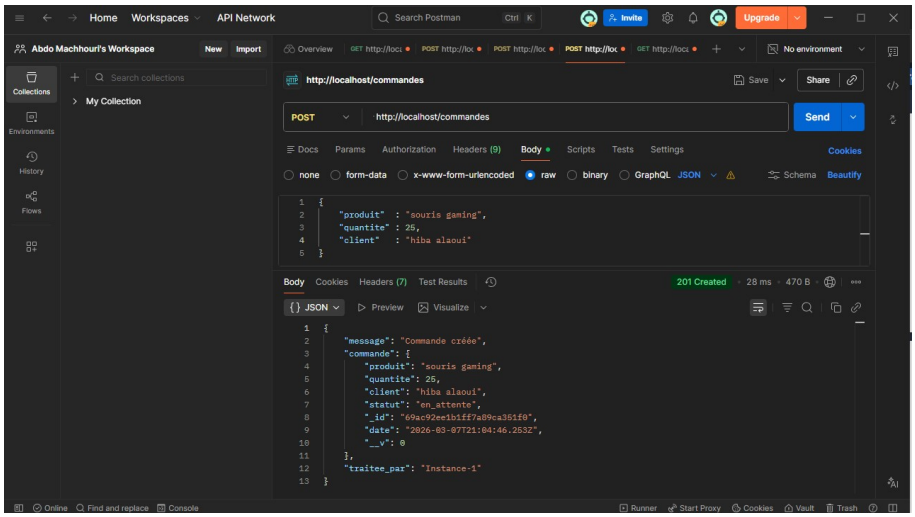

FIGURE 3.11 – Load Balancing - Requête 3 traitée par Instance-1

— Requête 4 → traitee_par : Instance-2

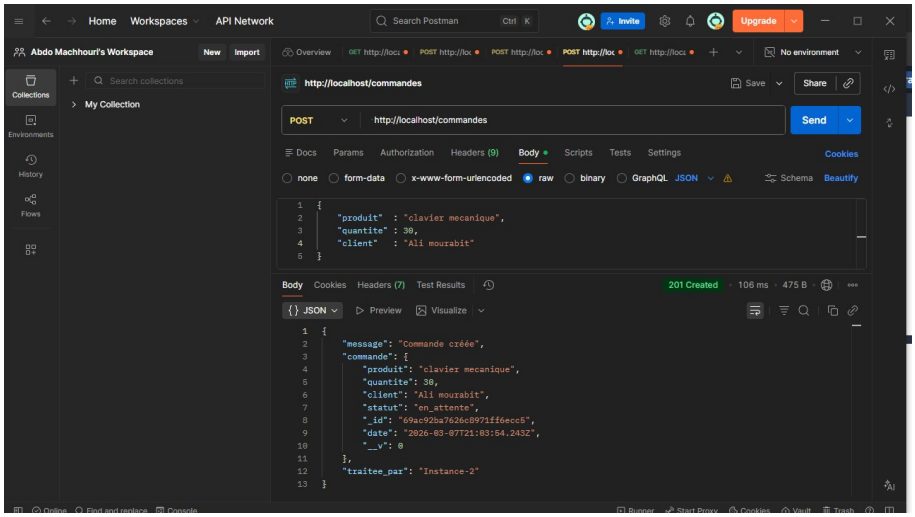

FIGURE 3.12 – Load Balancing - Requête 4 traitée par Instance-2

Cela confirme que Nginx répartit la charge en Round-robin.

3.8 Questions de compte-rendu

1. **Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?**
   Le composant qui reçoit en premier les requêtes est **Nginx (Load Balancer)**. Il utilise l'algorithme **Round-Robin** par défaut, qui distribue les requêtes de manière séquentielle et équitable entre les instances Node.js (app1 et app2), en alternant à chaque nouvelle requête.
2. **Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?**
   Si app1 tombe en panne, Nginx continue de fonctionner et redirige automatiquement toutes les requêtes vers app2. Cela est possible grâce au mécanisme de **health check passif** de Nginx, qui détecte les serveurs indisponibles et les exclut temporairement de la rotation, assurant ainsi la continuité du service.
3. **Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET (source Redis).**
   Le **1er GET** interroge directement la base de données **MongoDB**, ce qui implique une lecture sur disque et donc un temps de réponse plus élevé. Le **2ème GET** est servi depuis le cache **Redis**, qui stocke les résultats en mémoire vive. Redis étant beaucoup plus rapide que MongoDB, le temps de réponse est considérablement réduit, ce qui allège également la charge sur la base de données.
4. **Pourquoi invalide-t-on le cache Redis après un POST /commands ?**
   Le cache Redis est invalidé après un **POST /commands** afin de maintenir la **cohérence des données**. En effet, si le cache n'était pas invalidé, les prochaines lectures retourneraient des données obsolètes ne reflétant pas les nouvelles entrées insérées dans MongoDB. L'invalidation force donc une nouvelle lecture depuis MongoDB lors du prochain GET.

5. **En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?**
   RabbitMQ améliore la résilience du système en permettant un traitement **asynchrone** des événements. Les messages sont publiés dans une file d'attente et persistent même si le consommateur est temporairement indisponible. Lorsque le consommateur redémarre, il reprend le traitement des messages sans perte de données, garantissant ainsi la **fiabilité** et la **non-perte d'événements** dans le système.
6. **Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?**
   Il faut ajouter une nouvelle ligne dans le bloc `upstream backend` du fichier `nginx.conf` :

```
upstream backend {
    server app1:3001;
    server app2:3002;
    server app3:3003;  % Nouvelle instance ajoutee
}
```

   Il faudrait également déployer **app3** dans le `docker-compose.yml` en lui assignant le port **3003**, de la même manière que app1 et app2.